

TUGAS MODUL 3



Oleh :

Sandra Triana Nursyafri - 245150401111027

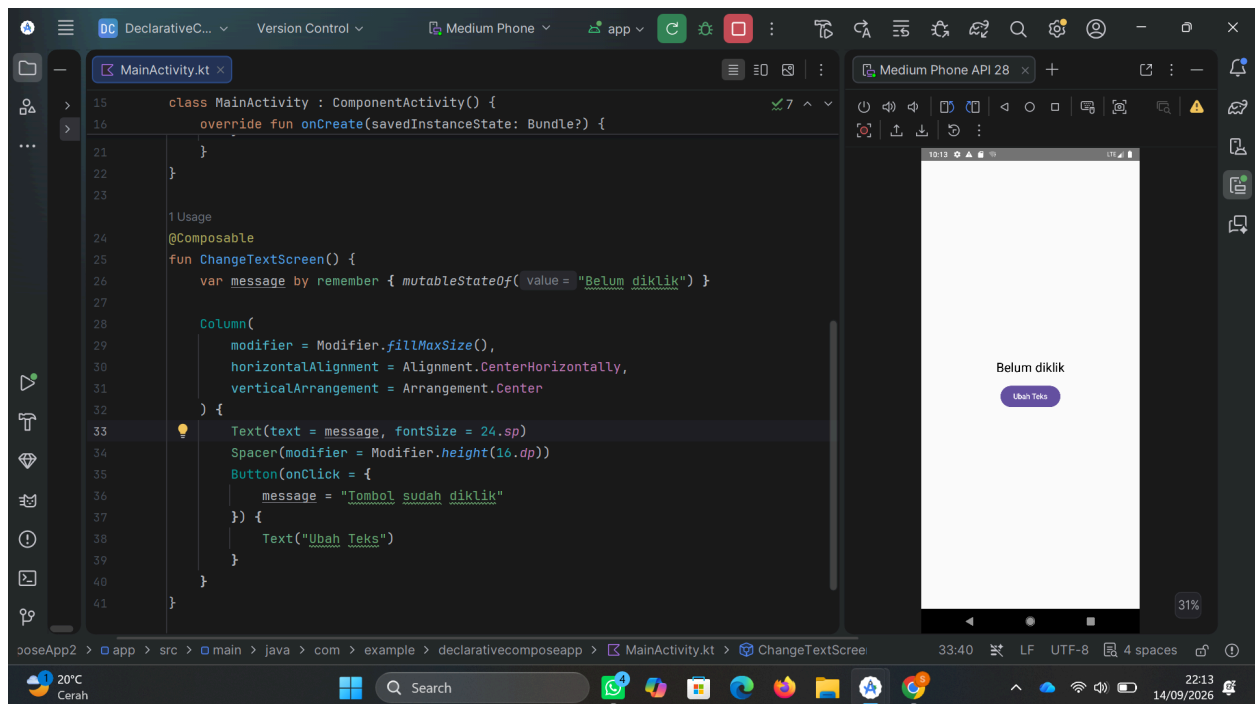
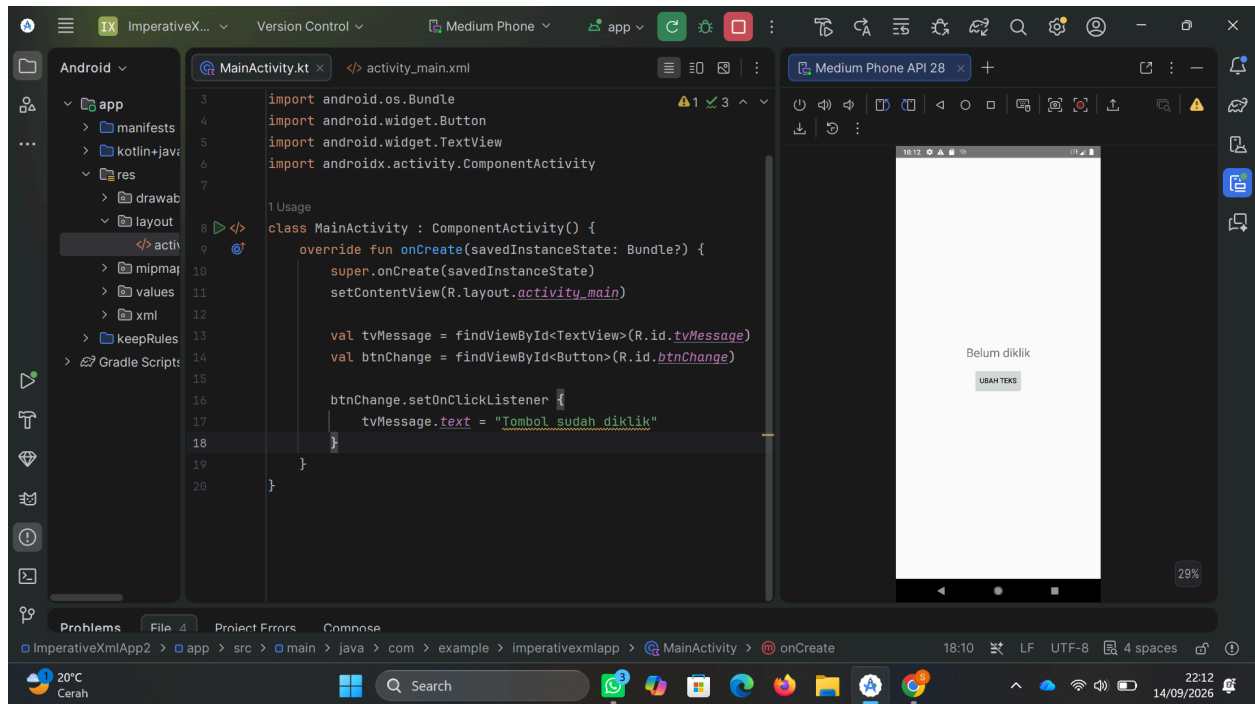
©

**FAKULTAS ILMU KOMPUTER
UNIVERSITAS BRAWIJAYA
MALANG
2026**

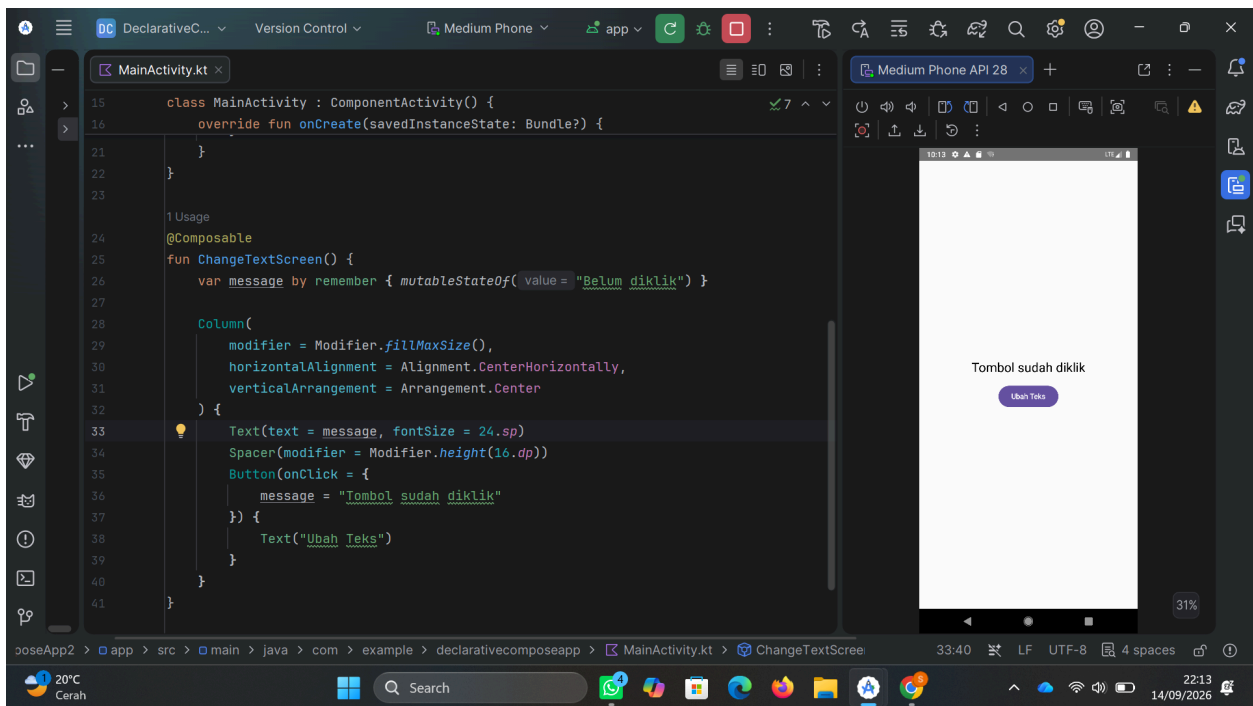
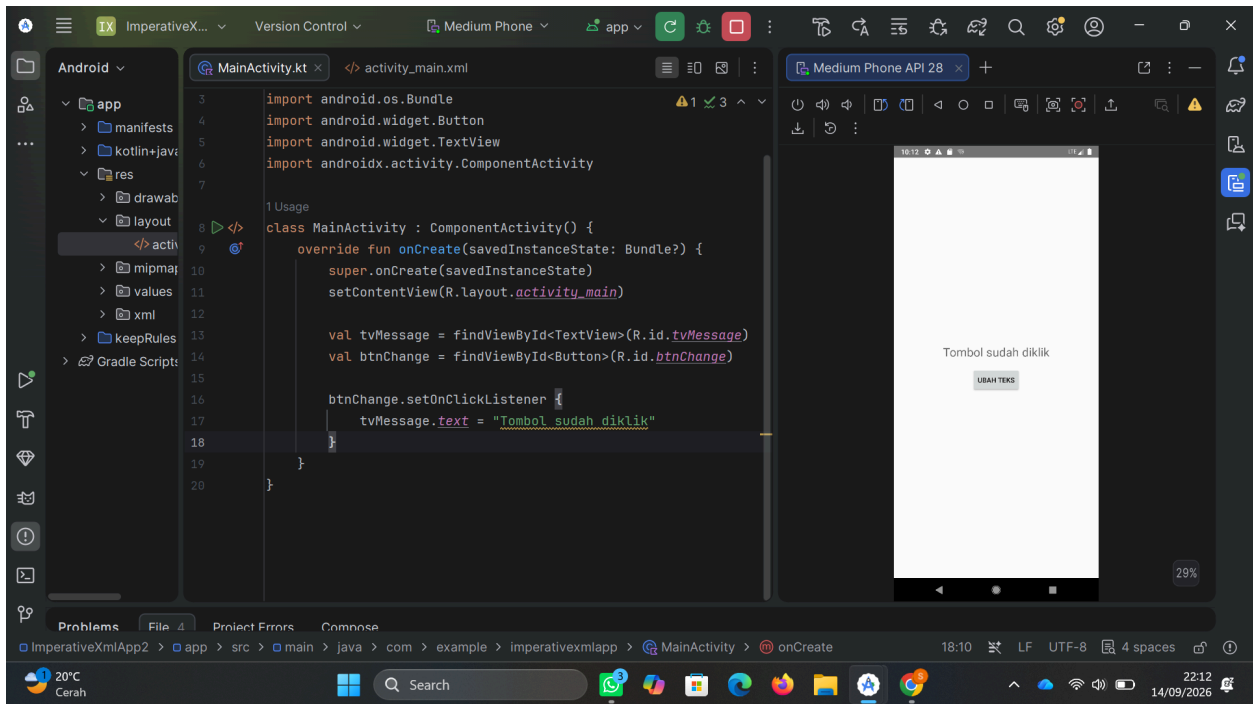
Aktivitas Percobaan

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Sebelum diklik



Setelah diklik



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.

Objek yang diubah langsung: tvMessage (TextView), baris tvMessage.text = "Tombol sudah diklik"

3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.
Yang diubah: state message, baris message = "Tombol sudah diklik"

4. Mengapa pada Compose tidak terdapat perintah seperti `tvMessage.text` atau `setText()`?

Karena Compose tidak memanipulasi View secara langsung Text hanya "membaca" nilai message. Saat message berubah, Compose recompose ulang Text, bukan memanggil method seperti `setText()`.

5. Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.

Karena Compose tidak memanipulasi View secara langsung — Text hanya "membaca" nilai message. Saat message berubah, Compose recompose ulang Text, bukan memanggil method seperti `setText()`.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Pola Compose: event (klik tombol) → ubah nilai message (state) → Compose mendeteksi state berubah → trigger recomposition pada Text yang bergantung pada message → UI otomatis diperbarui.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

Jika `var message = "Belum diklik"` (tanpa `remember { mutableStateOf(...) }`), maka mengubah message di dalam `onClick` tidak akan mengubah tampilan, karena Compose tidak tahu variabel biasa itu berubah tidak ada mekanisme observasi (observable state), jadi tidak ada recomposition yang dipicu.

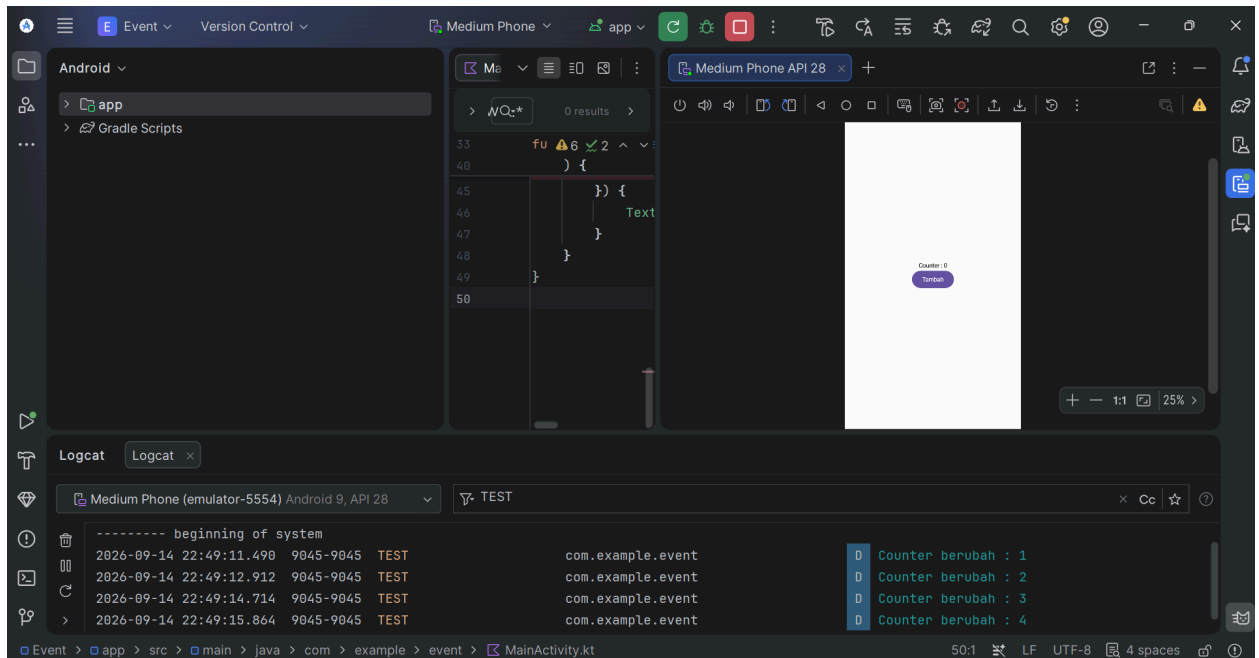
8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Imperatif (XML): Programmer harus memerintah sistem secara manual mengenai bagaimana cara mengubah komponen UI. Deklaratif (Compose): Programmer mendeskripsikan bagaimana bentuk UI berdasarkan state, dan framework yang akan memperbarui UI secara otomatis saat state berubah.

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST.

Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Muncul di LogCat (Tahap 1): Pesan LogCat bertambah (Counter berubah : 1, Counter berubah : 2, dst.).

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

teks layar tetap Counter : 0 (tidak berubah).

4. Apa yang menyebabkan hal tersebut terjadi?

count adalah variabel biasa, bukan State, sehingga Compose tidak memicu recomposition.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.

LogCat tetap mencatat nilai count yang bertambah.

6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

```
var count = 0
```

menjadi:

```
var count by remember { mutableStateOf(0) }
```

7. Berikut kode lengkap EventStateTest:

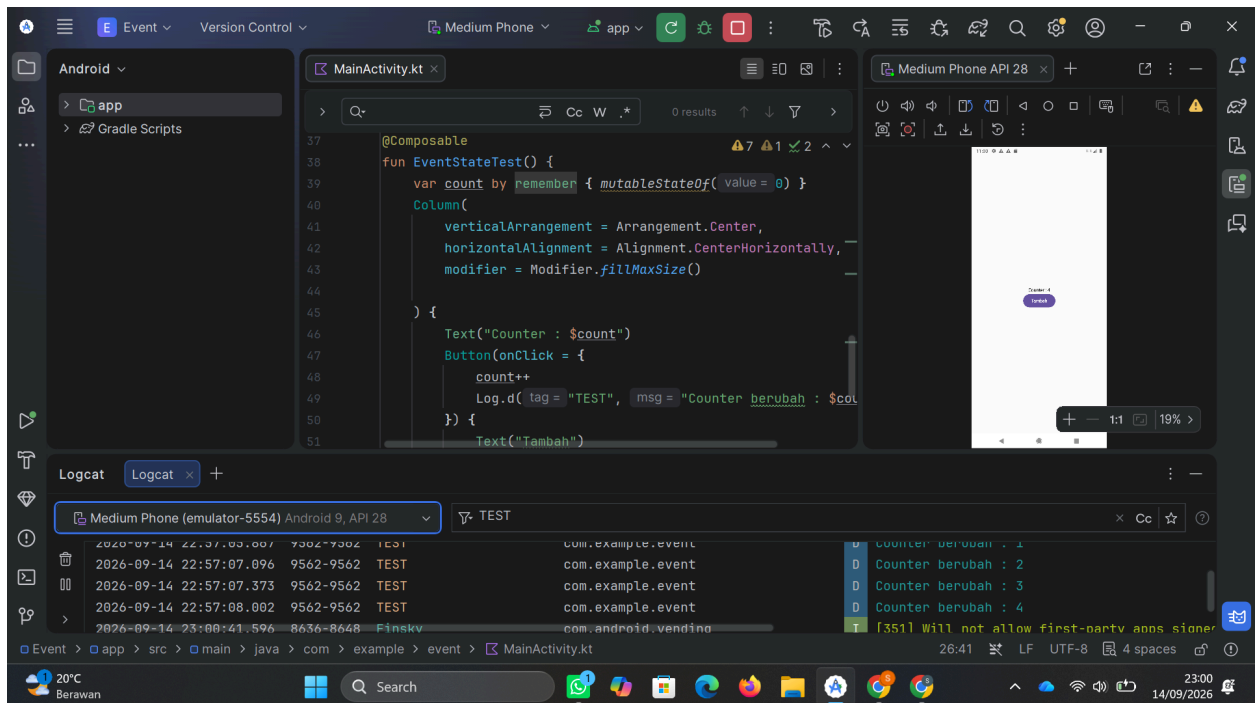
@Composable

```

fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
    }
}

```

8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

LogCat menampilkan pesan log dengan tag TEST yang mencetak isi teks terbaru setiap kali terjadi perubahan karakter

10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Karakter atau teks yang diketikkan pengguna akan langsung muncul dan diperbarui secara instan di dalam kotak OutlinedTextField di layar.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Ketika pengguna mengetik, komponen OutlinedTextField memicu callback event `onValueChange`. Di dalam callback tersebut, variabel state `text` diperbarui dengan karakter terbaru (`text = it`). Perubahan state `text` ini kemudian memicu recomposition sehingga OutlinedTextField digambar ulang dengan nilai parameter `value = text` yang baru

12. Apa yang dimaksud recomposition pada Compose?

Ketika pengguna mengetik, komponen OutlinedTextField memicu callback event `onValueChange`. Di dalam callback tersebut, variabel state `text` diperbarui dengan karakter terbaru (`text = it`). Perubahan state `text` ini kemudian memicu recomposition sehingga OutlinedTextField digambar ulang dengan nilai parameter `value = text` yang baru.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu `text` bertipe `string` pada Composable `EventStateTest`. Kemudian di dalam `Column` panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable `EventStateTest` menjadi:

```
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

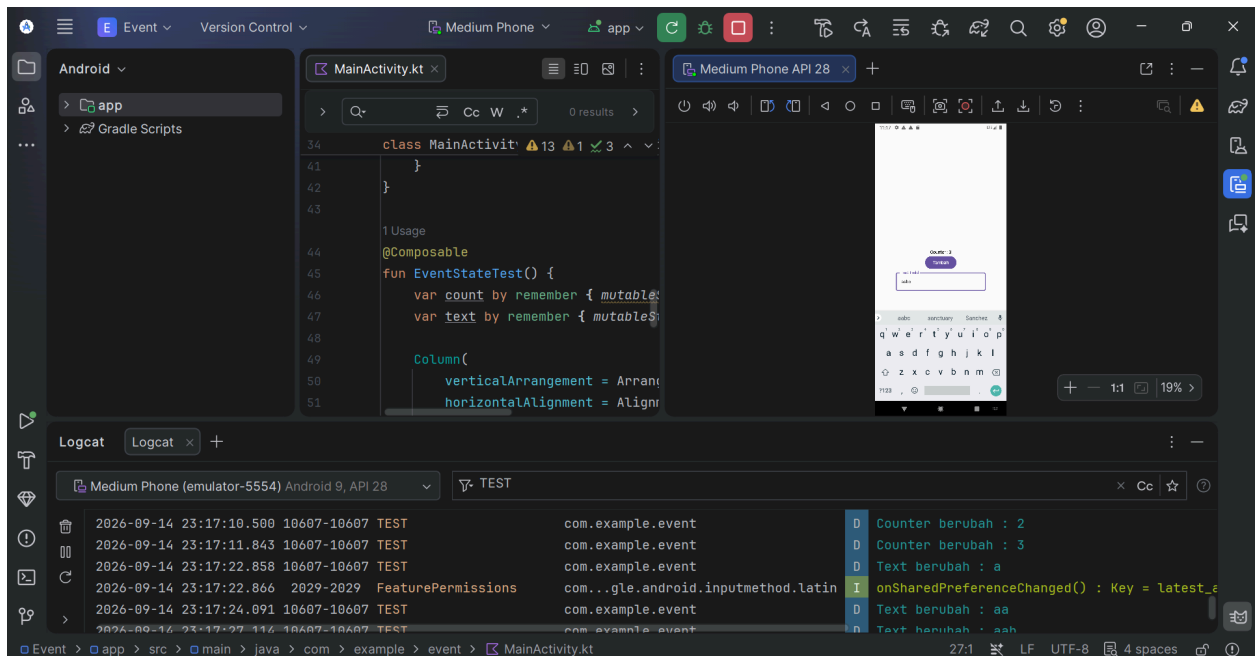
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
        OutlinedTextField(
            value = text,
            onValueChange = {
                text = it
            }
        )
    }
}
```

```

        Log.d("TEST", "Text berubah : $it")
    },
    label = { Text("Text Field") }
}
}
}

```

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?

LogCat menampilkan pesan log dengan tag TEST yang mencetak isi teks terbaru setiap kali terjadi perubahan karakter.

16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?

Ketika pengguna mengetik, komponen OutlinedTextField memicu callback event onValueChange. Di dalam callback tersebut, variabel state text diperbarui dengan karakter terbaru (text = it). Perubahan state text ini kemudian memicu recomposition sehingga OutlinedTextField digambar ulang dengan nilai parameter value = text yang baru.

17. Apa yang menyebabkan hal tersebut terjadi?

Ketika pengguna mengetik, komponen OutlinedTextField memicu callback event onValueChange. Di dalam callback tersebut, variabel state text diperbarui dengan karakter terbaru (text = it). Perubahan state text ini kemudian memicu recomposition

sehingga OutlinedTextField digambar ulang dengan nilai parameter value = text yang baru.

18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?

count: Menyimpan data bertipe integer untuk angka counter. text: Menyimpan data bertipe String untuk inputan karakter pada text field.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

onClick pada komponen Button. onChange pada komponen OutlinedTextField.

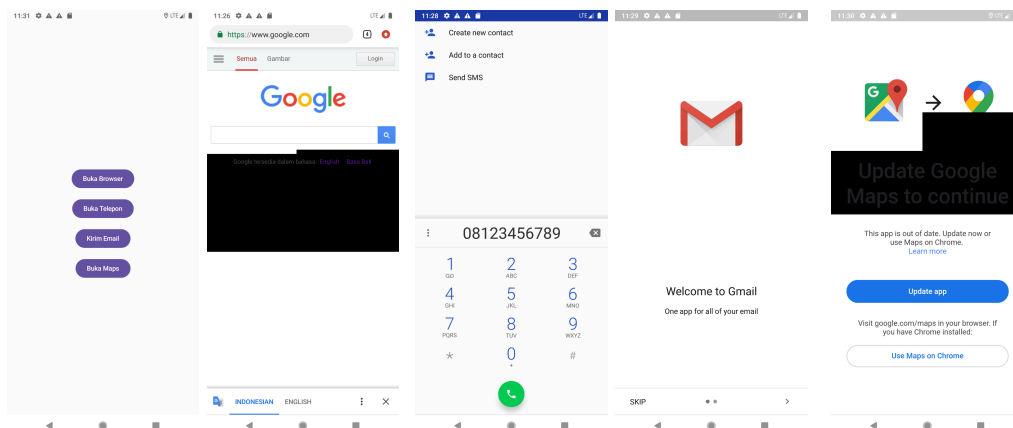
20. Kapan event - event tersebut dijalankan?

onClick: Dijalankan saat pengguna menekan atau mengetuk (tap) tombol "Tambah" di layar. onChange: Dijalankan setiap kali ada perubahan pada kotak teks, baik saat pengguna mengetikkan karakter baru, menempelkan (paste) teks, maupun menghapus karakter.

Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:
2. Klik Buka Browser
3. Klik Buka Telepon
4. Klik Kirim Email
5. Klik Buka Maps
6. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya

Halaman utama Buka browser Buka telepon Buka email Buka maps



7. Pada kode yang digunakan, terdapat beberapa action seperti:
8. Intent.ACTION_VIEW
9. Intent.ACTION_DIAL
10. Intent.ACTION_SENDTO
11. Apa perbedaan fungsi dari masing-masing action intent tersebut?

ACTION_VIEW: Menampilkan/membuka data (seperti URL web atau koordinat peta).

ACTION_DIAL: Membuka aplikasi telepon dan mengisi nomor tujuan.

ACTION_SENDTO: Mengirim pesan/email ke alamat tertentu.

12. Pada kode yang digunakan, terdapat beberapa data seperti:

13. data = Uri.parse("mailto:test@gmail.com")

14. data = Uri.parse("<https://www.google.com>")

15. data = Uri.parse("geo:-7.250445,112.768845")

16. Apa fungsi data pada intent ?

Sebagai URI target atau muatan spesifik (seperti link https:, nomor tel:, email mailto:, atau koordinat geo:) yang akan diproses oleh Intent.

17. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android

Android menggunakan kombinasi Action (tindakan) dan Data (subjek/skema URI) untuk memfilter dan menentukan aplikasi mana yang cocok untuk menjalankan tugas tersebut.